Week_09_Quiz-jeg2253

November 24, 2025

1 Week 9 Quiz

1.1 Jayat González Palomeras - jeg2253

1.1.1 Due Tue. Nov 25, 11:59pm

This quiz has two sections. In section 1, we'll practice scaling data and using PCA for dimensionality reduction. In section 2, we focus on NLP and TF-IDF methods.

1.1.2 Instructions

Replace the Name and UNI in cell above and the notebook filename

Replace all '_____' below using the instructions provided.

When completed, 1. make sure you've replaced Name and UNI in the first cell and filename 2. Kernel -> Restart & Run All to run all cells in order 3. Print Preview -> Print (Landscape Layout) -> Save to pdf 4. post pdf to GradeScope

2 PART 1

2.0.1 Load Standard Libraries

[1]:
```
# Import numpy, pandas, matplotlib.pyplot and seaborn
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns


# Set matplotlib to display inline
%matplotlib inline
```

2.0.2 Load the Dataset

[2]:
```
# Import load_breast_cancer from sklearn.datasets
from sklearn.datasets import load_breast_cancer

# Load the breast cancer dataset using the load_breast_cancer() function.
# Store in the variable 'cancer'.
```

```
cancer = load_breast_cancer()

# Create a new dataframe X with values from cancer.data (which is stored as a␣
 ↪numpy array)
#    and with columns named using cancer.feature_names (also a numpy array)
X = pd.DataFrame(cancer.data,columns=cancer.feature_names)

# For this quiz, only keep the first 10 features/columns
# Store the result back into X
X = X.iloc[:,:10]

# Assert that the shape of the dataframe is (569,10): 569 rows, 10 columns
assert X.shape == (569,10)
```

2.0.3 Calculate Summary Stats

[3]:
```
# The distribution of features in this dataset vary quite a bit, affecting PCA␣
 ↪performance.
# To get a sense of the difference, display the mean and standard deviation of␣
 ↪each feature.
# Use the .agg() function, which takes a list of strings describing the␣
 ↪functions to apply.
# Call .agg() on X
#   with the function names 'mean' and 'std'
#   transpose the dataframe using .T or .transpose()
#   and round to a precision of 2
X.agg(['mean', 'std']).T.round(2)
```

[3]:

	mean	std
mean radius	14.13	3.52
mean texture	19.29	4.30
mean perimeter	91.97	24.30
mean area	654.89	351.91
mean smoothness	0.10	0.01
mean compactness	0.10	0.05
mean concavity	0.09	0.08
mean concave points	0.05	0.04
mean symmetry	0.18	0.03
mean fractal dimension	0.06	0.01

2.0.4 Standardize the Data

[4]:
```
# Standardize the data to mean 0, standard deviation of 1 using sklearn␣
 ↪StandardScaler

#Import StandardScaler from sklearn
from sklearn.preprocessing import StandardScaler
```

```
# To standardize X use StandardScaler with default settings
#  do a fit_transform() on X
#  store in X_zscore
X_zscore = StandardScaler().fit_transform(X)

# Add feature names by creating a new DataFrame
#  containing X_zscore
#  with the same column names as the original dataframe X
#  store back into X_zscore
X_zscore = pd.DataFrame(X_zscore, columns = X.columns)

# assert that the mean is near 0 and standard deviation is near 1 for all␣
 ↪standardized features
assert X_zscore.mean().round(2).eq(0).all() and X_zscore.std().round(2).eq(1).
 ↪all()

# To visually confirm that all features have been standardized:
# Call .agg() on X_zscore
#   with the function names 'mean' and 'std'
#   transpose the dataframe using .T or .transpose()
#   and round to a precision of 2
X_zscore.agg(['mean', 'std']).T.round(2)
```

[4]:

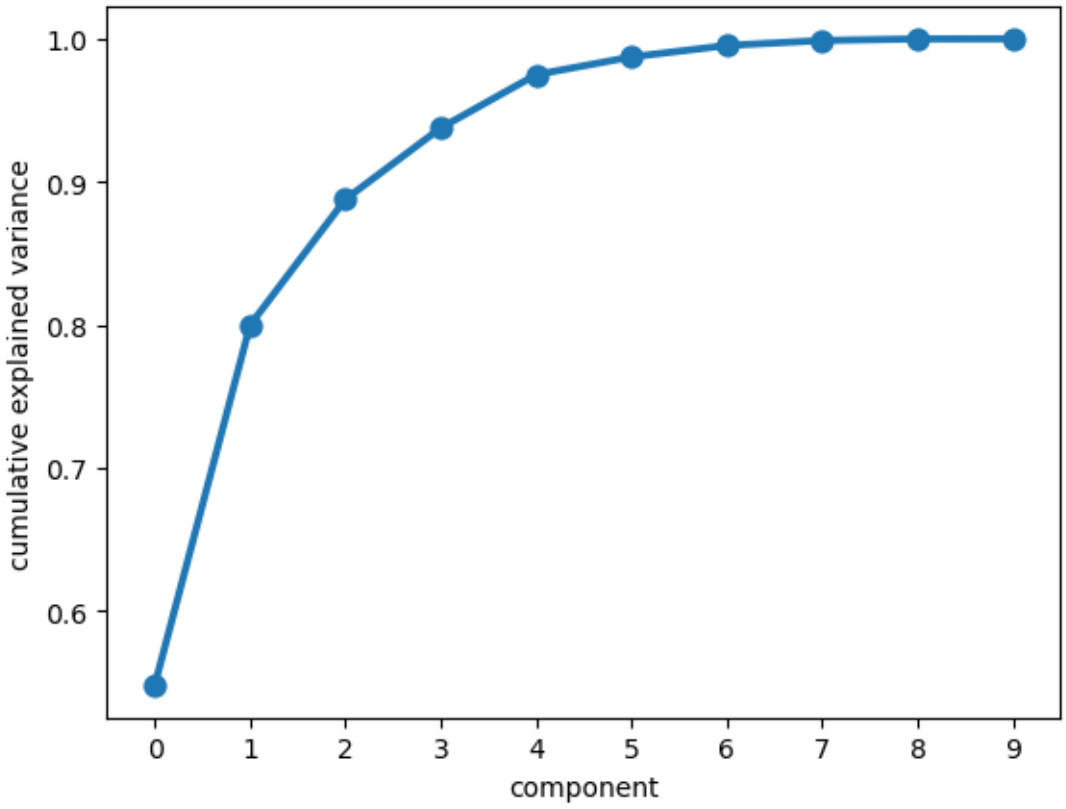
	mean	std
mean radius	-0.0	1.0
mean texture	-0.0	1.0
mean perimeter	-0.0	1.0
mean area	-0.0	1.0
mean smoothness	0.0	1.0
mean compactness	-0.0	1.0
mean concavity	-0.0	1.0
mean concave points	0.0	1.0
mean symmetry	-0.0	1.0
mean fractal dimension	-0.0	1.0

2.0.5 Show Variance Described by PCA

[5]:
```
# Import PCA from sklearn.
from sklearn.decomposition import PCA

# Fit a PCA model to X_zscore using PCA() with default parameters
#   and store in pca
pca = PCA().fit(X_zscore)

# Create a new DataFrame with 2 columns:
#   "component" with values 0 to the number of components in pca
```

```
#   "cumulative explained variance" with the .cumsum() of the␣
 ↪explained_variance_ratio_ in pca
#   store in df_var
df_var = pd.DataFrame({"component": np.arange(len(pca.
 ↪explained_variance_ratio_)),
                       "cumulative explained variance": pca.
 ↪explained_variance_ratio_.cumsum()})


# Use sns.pointplot() to plot the data from df_var with
#   "component" on the x-axis
#   "cumulative explained variance" on the y-axis
sns.pointplot(data= df_var, x="component", y="cumulative explained variance")

# Note that over 55% of the variance is explained by the first component
# Over 80% by the first 2 components
# Over 90% by the first 4 components
```

[5]: <Axes: xlabel='component', ylabel='cumulative explained variance'>



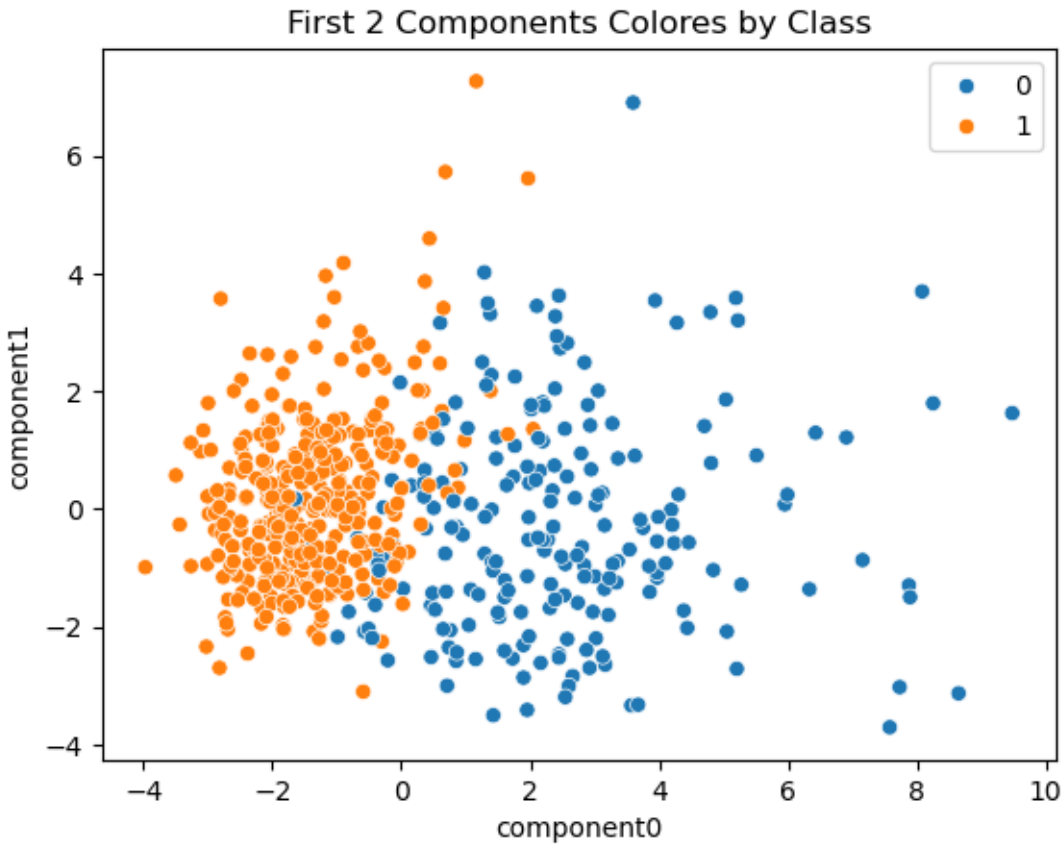
2.0.6 Transform Dataset using First 2 Components

[6]:
```
# Fit and transform X_zscore using a new PCA model with n_components=2
# Store the transformed dataset in X_pca
X_pca = PCA(n_components=2).fit_transform(X_zscore)

# Add feature names by creating a new DataFrame
#  containing X_pca
#  with the column names ['component0','component1']
#  store back into X_pca
X_pca = pd.DataFrame(X_pca, columns=['component0', 'component1'])

# Assert that the pca representation has the same number of rows (569) but now␣
 ↪2 columns
assert X_pca.shape == (569,2)
```

2.0.7 Plot the Reduced Representation

[7]:
```
# Using seaborn, create a scatterplot of the data in X_pca
#  with component0 on the x-axis
#  and component1 on the y-axis
#  Color the points by their class assignment by setting hue=cancer.target
#  Capture the returned axis in ax
ax = sns.scatterplot(data=X_pca, x='component0', y='component1', hue=cancer.
 ↪target)

# Set the title to 'First 2 Components Colored by Class' using ax
ax.set_title('First 2 Components Colores by Class')

# Note that we haven't used the cancer.target information to generate the pca␣
 ↪representation.
# We're coloring by cancer.target here to demonstrate that under this␣
 ↪transformation
#   a linear model will do a decent job of separating the classes
```

[7]:
```
Text(0.5, 1.0, 'First 2 Components Colores by Class')
```



3 PART 2

3.0.1 Load the Dataset

[8]:
```
# Import fetch_20newsgroups from sklearn.datasets
from sklearn.datasets import fetch_20newsgroups

# Load the dataset using fetch_20newsgroups().
#   Only fetch the two categories of interest using categories=['sci.
 ↪space','rec.autos']
# Store in the result into newsgroups
newsgroups = fetch_20newsgroups(categories=['sci.space','rec.autos'])

# Store the newsgroups.data as docs, newsgroups.target as y and newsgroups.
 ↪target_names as y_names
docs = newsgroups.data
y = newsgroups.target
y_names = newsgroups.target_names
```

```
# Print the number of observations by printing the length of docs
#  You should get 1187
len(docs)
```

[8]: 1187

[9]:
```
# Print the text of the first document in docs.
# Note: try printing both with and without the print() statement
#  with the print statement, linebreaks are parsed,
#  without, linebreaks are printed as excape characters
print(docs[0])
```

```
From: prb@access.digex.com (Pat)
Subject: Re: Proton/Centaur?
Organization: Express Access Online Communications USA
Lines: 15
NNTP-Posting-Host: access.digex.net


Well thank you dennis for your as usual highly detailed and informative
posting.

The question i have about the proton, is  could it be  handled at
one of KSC's spare pads, without major  malfunction,  or could it be
handled at kourou  or Vandenberg?

Now if it uses storables,  then  how long would it take for the russians
to equip something at cape york?

If  Proton were launched from a western site,  how would it compare to the
T4/centaur?   As i see it, it should lift  very close to the T4.

pat
```

[10]:
```
# Print the target value of the first document in y.
print(y[0])
```

```
1
```

[11]:
```
# Print the target_name of the first document using y_names and y.
print(y_names[y[0]])
```

```
sci.space
```

3.0.2 Use CountVectorizer to Convert To TF

[12]:
```
# Import CountVectorizer from sklearn
from sklearn.feature_extraction.text import CountVectorizer

# Initialize a CountVectorizer object. It should
#   lowercase all text,
#   include both unigrams and bigrams: ngram_range=(1,2)
#   exclude terms that occur in fewer than 10 documents: min_df=10
#   exclude terms that occur in more than 95% of documents: max_df=.95
# Store as cvect
cvect= CountVectorizer(lowercase=True, ngram_range= (1,2), min_df=10, max_df=0.
 ↪95)

# Fit cvect on docs and transform docs into their term frequency representation.
# Store as X_tf
X_tf = cvect.fit_transform(docs)

# Print the shape of X_tf.
# The number of rows should match the number of documents above
#  and the number of columns should be near 6000
print(X_tf.shape)
```

```
(1187, 5893)
```

[13]:
```
# Print out the last 5 terms in the learned vocabulary in cvect
#   using .get_feature_names_out() which returns a list of terms corresponding
#   to the order of the columns in X_tf
# They should all be related to zoos or zoology
cvect.get_feature_names_out()[-5:]
```

[13]:
```
array(['zoo', 'zoo toronto', 'zoology', 'zoology kipling',
       'zoology lines'], dtype=object)
```

[14]:
```
# The stopwords learned by cvect are stored as a set in cvect.stop_words_
# We'd like to print out a small subset of these terms.
# One way to get a subset of a set is to treat it as a list.
# First, convert the stop_words_ set to a list.
# Store as stop_words_list
stop_words_list= list(cvect.stop_words_)

# Print out the first 5 elements in stop_words_list.
# Note that, since a set is unordered,
#  there is no meaning to the ordering of these terms and they may vary over␣
 ↪runs.
stop_words_list[:5]
```

[14]:
```
['controllers and',
 'sounds better',
 'contact pete',
 'that design',
 'city organization']
```

3.0.3 Calculate Mean CV Accuracy Using RandomForestClassifier

[15]:
```
# Import cross_val_score from sklearn
from sklearn.model_selection import cross_val_score

# Import RandomForestClassifier from sklearn
from sklearn.ensemble import RandomForestClassifier

# Get a set of 5-fold CV scores using
#  a RandomForestClassifier
#   with 50 trees
#   and n_jobs=-1 all other settings default
#   and the full dataset X_tf and y
# Store as cv_scores
cv_scores = cross_val_score(RandomForestClassifier(n_estimators=50, n_jobs=-1),
                            X_tf, y, cv=5)

# Print the mean of these cv_scores rounded to a precision of 2.
#  The mean accuracy should be above .9
round(cv_scores.mean(),2)
```

[15]:
```
0.97
```

3.0.4 Optional: Find Important Features

[16]:
```
# CountVectorizer stores the feature names (terms in the vocabulary) in two␣
 ↪ways:
#  1. as a dictionary of term:colum_index pairs, accessed via cvect.vocabulary_
#  2. as a list of terms, in column index order, accessed via cvect.
 ↪get_feature_names_out()
#
# We can get the indices of the most important features by training a new␣
 ↪RandomForestClassifier on X_tf,y
#  and accessing .feature_importances_
#
# Using some combination of the above data-structures,
#  print out the top 10 terms in the vocabulary
#  ranked by the feature importances learned by a RandomForestClassifier with␣
 ↪50 trees
#
```

```
# The terms you find will likely not be surprising given the document␣
 ↪categories.
rf = RandomForestClassifier(n_estimators=50, n_jobs=-1).fit(X_tf, y)
importances = rf.feature_importances_

top_idx = np.argsort(importances)[-10:][::-1]

top_terms=cvect.get_feature_names_out()[top_idx]
top_terms
```

[16]:
```
array(['car', 'space', 'orbit', 'cars', 'nasa', 'nasa gov', 'the car',
       'my', 'launch', 'moon'], dtype=object)
```

[ ]:
```

```